

Databricks Milestone 2

All Code-Based MCQs — Exercises 1–4

92 exam-style questions with code, options, answers, and explanations

How to use: Cover the answer, solve the snippet, then check the explanation. Focus on why the other options are wrong.

Exercise 1 — Java Fundamentals, Constructors, Overloading, Memory & Operators

Q1. *send("25") overload*

```
static void send(int v){System.out.println("int");}  
static void send(String v){System.out.println("String");}  
send("25");
```

- A) int
- B) String
- C) Compile error
- D) Runtime error

Answer: B) String

Explanation: "25" is a String literal.

Q2. *use(int) overload*

```
static void use(long v){System.out.println("long");}  
static void use(Integer v){System.out.println("Integer");}  
int n=5; use(n);
```

- A) Integer
- B) long
- C) Both
- D) Compile error

Answer: B) long

Explanation: int→long is primitive widening; int→Integer is boxing. Widening is preferred here.

Q3. *Integer argument*

```
static void use(long v){System.out.println("long");}  
static void use(Integer v){System.out.println("Integer");}  
Integer n=5; use(n);
```

- A) long
- B) Integer
- C) Both
- D) Compile error

Answer: B) Integer

Explanation: The argument is already an Integer.

Q4. *var numeric promotion*

```
int a=2; long b=3L;  
var result=a+b;
```

- A) int
- B) long
- C) double
- D) Integer

Answer: B) long

Explanation: int is promoted to long.

Q5. numeric output

```
int a=10; long b=20L;  
var result=a+b;  
System.out.println(result);
```

- A) 10
- B) 20
- C) 30
- D) Error

Answer: C) 30

Explanation: 10+20L gives 30L.

Q6. this() constructor order

```
Test(){this(10);System.out.println("A");}  
Test(int x){System.out.println("B");}  
new Test();
```

- A) A B
- B) B A
- C) A
- D) B

Answer: B) B A

Explanation: this(10) calls the parameterized constructor first.

Q7. this() must be first

```
Test(){System.out.println("Hello");this(10);}  
Test(int x){}
```

- A) Hello Number
- B) Number Hello
- C) Compile error
- D) Runtime error

Answer: C) Compile error

Explanation: A constructor call using this(...) must be the first statement.

Q8. final field initialization

```
class Test{final int id; Test(){id=101;}}  
Test t=new Test(); System.out.println(t.id);
```

- A) 0
- B) 101
- C) Error
- D) null

Answer: B) 101

Explanation: The final field is assigned in the constructor.

Q9. reassign final

```
class Test{final int id=10;}  
Test t=new Test(); t.id=20;
```

- A) 20
- B) 10
- C) Compile error
- D) Runtime error

Answer: C) Compile error

Explanation: A final variable cannot be reassigned.

Q10. static field

```
class Test{static String company="Learnlytica";}
System.out.println(Test.company);
```

- A) Test
- B) Learnlytica
- C) null
- D) Error

Answer: B) Learnlytica

Explanation: static belongs to the class.

Q11. static field is shared

```
class Test{static int count=10;}
Test a=new Test(); Test b=new Test();
a.count=20; System.out.println(b.count);
```

- A) 10
- B) 20
- C) 0
- D) Error

Answer: B) 20

Explanation: Both objects access the same static field.

Q12. autoboxing

```
Integer quantity=7;
System.out.println(quantity);
```

- A) Error
- B) 7 is boxed to Integer
- C) Integer→int
- D) Runtime error

Answer: B) 7 is boxed to Integer

Explanation: int→Integer is autoboxing.

Q13. unboxing

```
Integer x=10; int y=x;
System.out.println(y);
```

- A) Error
- B) 10
- C) null
- D) Runtime error

Answer: B) 10

Explanation: Integer→int is unboxing.

Q14. object allocation

```
Customer c=new Customer();
```

- A) Stack
- B) Heap
- C) Bytecode
- D) Method parameter

Answer: B) Heap

Explanation: The object created by new is conceptually on the heap.

Q15. main stack frame

```
public static void main(String[] args){  
    int x=10; Customer c=new Customer();  
}
```

- A) Permanent
- B) Removed when main returns
- C) Moves to heap
- D) Becomes bytecode

Answer: B) Removed when main returns

Explanation: A method's stack frame is removed when the method returns.

Q16. short-circuit &&

```
Customer obj=null;  
if(obj!=null && obj.isValid()) System.out.println("Valid");
```

- A) Valid
- B) NullPointerException
- C) isValid() is not executed
- D) Compile error

Answer: C) isValid() is not executed

Explanation: The false left operand short-circuits &&.

Q17. short-circuit side effect

```
int x=10;  
if(x<5 && ++x>10) System.out.println("Yes");  
System.out.println(x);
```

- A) Yes,11
- B) 10
- C) 11
- D) Yes,10

Answer: B) 10

Explanation: x<5 is false, so ++x is skipped.

Q18. short-circuit ||

```
int x=10;  
if(x>5 || ++x>20) System.out.println("Yes");  
System.out.println(x);
```

- A) Yes,11
- B) Yes,10
- C) No,11
- D) 10

Answer: B) Yes,10

Explanation: The true left operand short-circuits ||.

Q19. operator precedence

```
int answer=2+3*4;  
System.out.println(answer);
```

- A) 20
- B) 14
- C) 24
- D) 10

Answer: B) 14

Explanation: Multiplication is evaluated before addition.

Q20. parentheses

```
int answer=(2+3)*4;
System.out.println(answer);
```

- A) 14
- B) 20
- C) 24
- D) 9

Answer: B) 20

Explanation: Parentheses are evaluated first.

Q21. combined overload + precedence

```
static void use(long x){System.out.println("long");}
static void use(Integer x){System.out.println("Integer");}
int n=5; use(n);
int x=2+3*4; System.out.println(x);
```

- A) Integer / 14
- B) long / 14
- C) Integer / 20
- D) long / 20

Answer: B) long / 14

Explanation: Widening selects long; precedence gives 14.

Q22. combined constructor + final

```
final int id;
Test(){this(100);System.out.println("A");}
Test(int id){this.id=id;System.out.println("B");}
Test t=new Test(); System.out.println(t.id);
```

- A) A B 100
- B) B A 100
- C) B 100 A
- D) Error

Answer: B) B A 100

Explanation: The chained constructor prints B first, then A, then id.

Q23. shared static count

```
static int count=5;
Test a=new Test(); Test b=new Test();
a.count++; b.count++;
System.out.println(Test.count);
```

- A) 5
- B) 6
- C) 7
- D) 8

Answer: C) 7

Explanation: The static field is shared by both objects.

Q24. big combined Exercise 1

```
Customer obj=null;
if(obj!=null && obj.isValid()) System.out.println("Valid");
int n=5; use(n);
new Exercise1();
int answer=2+3*4;
System.out.println(answer);
System.out.println(Exercise1.company);
```

A) Checking / Integer / Default / Parameterized / 14 / Learnlytica

B) long / Parameterized / Default / 14 / Learnlytica

C) Integer / Default / Parameterized / 20 / Learnlytica

D) long / Default / Parameterized / 14 / Learnlytica

Answer: B) long / Parameterized / Default / 14 / Learnlytica

Explanation: && skips the null call; widening selects long; this(...) chains to the parameterized constructor; 2+3*4=14.

Exercise 2 — Collections, Generics, Iterators & Regex

Q25. generic List get(0)

```
List<String> names=new ArrayList<>();  
names.add("Mira"); names.add("Anand");  
System.out.println(names.get(0));
```

- A) Anand
- B) Mira
- C) 0
- D) Error

Answer: B) Mira

Explanation: List indexing starts at 0.

Q26. generic type safety

```
List<String> names=new ArrayList<>();  
names.add("Mira"); names.add(100);
```

- A) Converts to String
- B) Compile error
- C) Runtime error
- D) Both accepted

Answer: B) Compile error

Explanation: List<String> only accepts String elements.

Q27. List duplicates

```
List<String> cities=new ArrayList<>();  
cities.add("Chennai"); cities.add("Chennai");  
System.out.println(cities.size());
```

- A) 0
- B) 1
- C) 2
- D) Error

Answer: C) 2

Explanation: List allows duplicates.

Q28. Set duplicates

```
Set<String> cities=new HashSet<>();  
cities.add("Chennai"); cities.add("Chennai");  
System.out.println(cities.size());
```

- A) 0
- B) 1
- C) 2
- D) Error

Answer: B) 1

Explanation: Set does not retain duplicate elements.

Q29. List vs Set

```
List<String> list; Set<String> set;
```

- A) List disallows duplicates
- B) Set allows duplicates
- C) List allows duplicates; Set does not
- D) Both remove duplicates

Answer: C) List allows duplicates; Set does not

Explanation: This is the core distinction.

Q30. HashSet custom objects

```
Set<CustomerKey> s=new HashSet<>();
s.add(new CustomerKey(101,"Mira"));
s.add(new CustomerKey(101,"Mira"));
System.out.println(s.size());
```

- A) 0
- B) 1
- C) 2
- D) Error

Answer: B) 1

Explanation: With consistent equals() and hashCode(), logically equal objects are one set element.

Q31. equals/hashCode rule

```
// equals() says two objects are equal.
// What must be true about hashCode()?
```

- A) Different hashes required
- B) Same hash code required
- C) No relation
- D) Hash code must be zero

Answer: B) Same hash code required

Explanation: Equal objects must produce the same hash code.

Q32. equals same data

```
CustomerKey a=new CustomerKey(101,"Mira");
CustomerKey b=new CustomerKey(101,"Mira");
System.out.println(a.equals(b));
```

- A) true
- B) false
- C) Error
- D) Exception

Answer: A) true

Explanation: The provided equals() compares id and name.

Q33. == versus equals()

```
CustomerKey a=new CustomerKey(101,"Mira");
CustomerKey b=new CustomerKey(101,"Mira");
System.out.println(a==b);
System.out.println(a.equals(b));
```

- A) true / true
- B) false / true
- C) true / false
- D) false / false

Answer: B) false / true

Explanation: They are different objects, but their logical state is equal.

Q34. HashMap lookup

```
Map<CustomerKey,String> map=new HashMap<>();
map.put(new CustomerKey(201,"Arun"),"Premium");
System.out.println(map.get(new CustomerKey(201,"Arun")));
```

- A) null
- B) Premium
- C) Arun
- D) Error

Answer: B) Premium

Explanation: Consistent equals/hashCode lets the logically equivalent key find the value.

Q35. *HashMap inconsistent equality*

```
map.put(new CustomerKey(201,"Arun"),"Premium");  
map.get(new CustomerKey(201,"Arun"));
```

- A) Strings unsupported
- B) Lookup may fail
- C) Key becomes String
- D) Always compile error

Answer: B) Lookup may fail

Explanation: Hash-based lookup depends on correct equality and hashing.

Q36. *ListIterator.next()*

```
ListIterator<String> it=List.of("A","B").listIterator();  
System.out.println(it.next());
```

- A) A
- B) B
- C) null
- D) Error

Answer: A) A

Explanation: The iterator initially points before A.

Q37. *two next() calls*

```
ListIterator<String> it=List.of("A","B").listIterator();  
String x=it.next(); String y=it.next();  
System.out.println(x); System.out.println(y);
```

- A) A / A
- B) B / A
- C) A / B
- D) B / B

Answer: C) A / B

Explanation: Successive next() calls return A then B.

Q38. *ListIterator.previous()*

```
ListIterator<String> it=List.of("A","B").listIterator();  
it.next(); it.next();  
System.out.println(it.previous());
```

- A) A
- B) B
- C) null
- D) Error

Answer: B) B

Explanation: After moving past B, previous() returns B.

Q39. *ListIterator full sequence*

```
ListIterator<String> it=List.of("A","B").listIterator();  
String a=it.next(); String b=it.next(); String c=it.previous();  
System.out.println(a+b+c);
```

- A) ABC
- B) ABB
- C) ABA
- D) BAA

Answer: B) ABB

Explanation: The values are A, B, B.

Q40. *Iterator.remove()*

```
List<Integer> n=new ArrayList<>(Arrays.asList(10,15,20,25));
Iterator<Integer> it=n.iterator();
while(it.hasNext()){
    int v=it.next();
    if(v%2!=0) it.remove();
}
System.out.println(n);
```

- A) [10,15,20,25]
- B) [15,25]
- C) [10,20]
- D) []

Answer: C) [10,20]

Explanation: remove() deletes the last value returned by next(); odd values are removed.

Q41. *remove() target*

```
int value=it.next();
it.remove();
```

- A) First element
- B) Last element returned by next()
- C) Next element
- D) All matches

Answer: B) Last element returned by next()

Explanation: That is the iterator.remove() rule.

Q42. *remove() before next()*

```
Iterator<Integer> it=numbers.iterator();
it.remove();
```

- A) Removes first
- B) Removes last
- C) No valid element to remove
- D) Clears list

Answer: C) No valid element to remove

Explanation: remove() needs a valid element returned by next().

Q43. *? extends accepted types*

```
static void printNumbers(List<? extends Number> values) { }
```

- A) List<Integer> only
- B) List<Double> only
- C) List<Long> only
- D) All of these

Answer: D) All of these

Explanation: The wildcard permits an unknown subtype of Number.

Q44. *? extends read*

```
static void printNumbers(List<? extends Number> values) {
    Number n=values.get(0);
    System.out.println(n);
}
```

- A) Valid
- B) Invalid
- C) Only Integer
- D) Runtime error

Answer: A) Valid

Explanation: Values can safely be read as Number.

Q45. ? extends add

```
static void addNumber(List<? extends Number> values) {  
    values.add(4);  
}
```

- A) 4 added
- B) Compile error
- C) Runtime exception
- D) Converts to Number

Answer: B) Compile error

Explanation: The actual list might be List<Double>, so adding an Integer is unsafe.

Q46. wildcard meaning

```
List<? extends Number>
```

- A) Only Number
- B) Unknown subtype of Number
- C) Any Object
- D) Only Integer

Answer: B) Unknown subtype of Number

Explanation: ? is an unknown type within the upper bound.

Q47. regex find()

```
Matcher m=Pattern.compile("\\d+").matcher("A12B34");  
int count=0; while(m.find()) count++;  
System.out.println(count);
```

- A) 0
- B) 1
- C) 2
- D) 4

Answer: C) 2

Explanation: find() finds 12 and 34.

Q48. find() meaning

```
Pattern.compile("\\d+").matcher("ABC123XYZ").find();
```

- A) Whole input digits
- B) Matching substring exists
- C) Input empty
- D) Regex valid

Answer: B) Matching substring exists

Explanation: find() searches for a matching part.

Q49. matches()

```
Matcher m=Pattern.compile("\\d+").matcher("A12");  
System.out.println(m.matches());
```

- A) true
- B) false
- C) 12
- D) Error

Answer: B) false

Explanation: matches() requires the entire input to match.

Q50. find vs matches

```
String input="A12"; Pattern p=Pattern.compile("\\d+");
System.out.println(p.matcher(input).find());
System.out.println(p.matcher(input).matches());
```

- A) true / true
- B) true / false
- C) false / true
- D) false / false

Answer: B) true / false

Explanation: find() locates 12; matches() rejects A12 as a full digit string.

Q51. regex full match

```
Matcher m=Pattern.compile("\\d+").matcher("12345");
System.out.println(m.matches());
```

- A) true
- B) false
- C) 12345
- D) Error

Answer: A) true

Explanation: The entire input is digits.

Q52. regex find count

```
Matcher m=Pattern.compile("\\d+").matcher("10A20B300");
int count=0; while(m.find()) count++;
System.out.println(count);
```

- A) 1
- B) 2
- C) 3
- D) 5

Answer: C) 3

Explanation: The matches are 10, 20, and 300.

Q53. List to Set

```
List<String> list=new ArrayList<>();
list.add("A"); list.add("A"); list.add("B");
Set<String> set=new HashSet<>(list);
System.out.println(list.size());
System.out.println(set.size());
```

- A) 2 / 2
- B) 3 / 3
- C) 3 / 2
- D) 2 / 3

Answer: C) 3 / 2

Explanation: List keeps both As; Set removes the duplicate.

Q54. Iterator + wildcard

```
List<Integer> values=new ArrayList<>(Arrays.asList(10,15,20,25));
Iterator<Integer> it=values.iterator();
while(it.hasNext()){int v=it.next();if(v%2!=0)it.remove();}
List<? extends Number> numbers=values;
System.out.println(numbers.get(0)); System.out.println(numbers.size());
```

A) 10 / 2

B) 15 / 2

C) 10 / 4

D) Error

Answer: A) 10 / 2**Explanation:** After removal the list is [10,20]; wildcard permits reading.**Q55. HashSet three objects**

```
Set<CustomerKey> s=new HashSet<>();
s.add(new CustomerKey(101,"Mira"));
s.add(new CustomerKey(101,"Mira"));
s.add(new CustomerKey(102,"Mira"));
System.out.println(s.size());
```

A) 1

B) 2

C) 3

D) 0

Answer: B) 2**Explanation:** The first two are equal; the third differs in id.**Q56. combined regex**

```
String input="A12B34C"; Pattern p=Pattern.compile("\\d+");
Matcher m=p.matcher(input); int count=0;
while(m.find())count++;
System.out.println(count);
System.out.println(p.matcher(input).matches());
```

A) 2 / true

B) 2 / false

C) 3 / false

D) 1 / true

Answer: B) 2 / false**Explanation:** find() finds two digit groups; matches() requires the whole string to be digits.

Exercise 3 — OOP, Polymorphism, Interfaces, Serialization & I/O

Q57. runtime polymorphism

```
class Animal{void speak(){System.out.println("Animal");}}
class Dog extends Animal{void speak(){System.out.println("Dog");}}
Animal a=new Dog(); a.speak();
```

- A) Animal
- B) Dog
- C) Error
- D) Runtime error

Answer: B) Dog

Explanation: The actual object is Dog, so Dog.speak() executes.

Q58. actual object determines override

```
Animal animal=new Dog(); animal.speak();
```

- A) Variable name
- B) Reference type only
- C) Actual object
- D) Constructor name

Answer: C) Actual object

Explanation: Overridden method dispatch occurs at runtime based on the actual object.

Q59. @Override purpose

```
class Dog extends Animal{
@Override void speak(){ }
}
```

- A) Makes static
- B) Indicates/checks an override
- C) Makes final
- D) Makes private

Answer: B) Indicates/checks an override

Explanation: @Override marks an intended overriding method.

Q60. composition relationship

```
class Engine{}
class Car{private Engine engine;
Car(Engine engine){this.engine=engine;}}
```

- A) Car IS-A Engine
- B) Car HAS-A Engine
- C) Engine IS-A Car
- D) Car extends Engine

Answer: B) Car HAS-A Engine

Explanation: Car contains/uses an Engine object.

Q61. composition call

```
class Engine{void start(){System.out.println("Engine");}}
class Car{private Engine e; Car(Engine e){this.e=e;}
void start(){e.start();}}
new Car(new Engine()).start();
```

- A) Inherits start
- B) Calls Engine.start
- C) Error
- D) Engine inherits Car

Answer: B) Calls Engine.start

Explanation: Car delegates to its contained Engine.

Q62. encapsulation

```
class BankAccount{
private double balance;
void deposit(double a){if(a>0)balance+=a;}
double getBalance(){return balance;}
}
```

- A) Private state + controlled methods
- B) Static state
- C) Final state
- D) Inheritance

Answer: A) Private state + controlled methods

Explanation: Private state and controlled methods demonstrate encapsulation.

Q63. abstract class

```
abstract class BaseService{
abstract void run();
final void policy(){System.out.println("Policy");}
}
```

- A) Directly instantiable
- B) Abstract + concrete methods
- C) All methods abstract
- D) Cannot contain concrete methods

Answer: B) Abstract + concrete methods

Explanation: Abstract classes can contain both abstract and implemented methods.

Q64. instantiate abstract class

```
abstract class BaseService{abstract void run();}
BaseService b=new BaseService();
```

- A) Works
- B) Compile error
- C) Runtime exception
- D) Auto-implements run

Answer: B) Compile error

Explanation: An abstract class cannot be directly instantiated.

Q65. interface implementation

```
interface Exportable{void export();}  
class ReportService implements Exportable{  
    public void export(){System.out.println("Export");}  
}
```

- A) extends Exportable
- B) implements Exportable
- C) Exportable extends class
- D) Error

Answer: B) implements Exportable

Explanation: A class implements an interface.

Q66. extends one, implements many

```
class ReportService extends BaseService  
implements Exportable, Auditable { }
```

- A) Two classes
- B) One class + two interfaces
- C) Two classes + one interface
- D) Three classes

Answer: B) One class + two interfaces

Explanation: A concrete class extends one class and can implement multiple interfaces.

Q67. multiple inheritance

```
class ReportService implements A,B {  
    class Other extends A implements B,C { }
```

- A) Neither
- B) First only
- C) Second only
- D) Both

Answer: D) Both

Explanation: Both declarations follow one superclass/multiple interfaces.

Q68. final method

```
class Base{final void run(){}  
class Child extends Base{void run(){} }
```

- A) Valid override
- B) Base executes
- C) Compile error
- D) Runtime error

Answer: C) Compile error

Explanation: A final method cannot be overridden.

Q69. final class

```
final class Animal{  
    class Dog extends Animal{ }
```

- A) Valid
- B) Compile error
- C) Runtime error
- D) Constructor issue

Answer: B) Compile error

Explanation: A final class cannot be extended.

Q70. private method and overriding

```
class Parent{private void run(){}  
class Child extends Parent{void run(){}}
```

- A) Overrides
- B) Does not override Parent's private method
- C) Override only with annotation
- D) Override if static

Answer: B) Does not override Parent's private method

Explanation: Private superclass methods are not overridden in the normal inheritance sense.

Q71. protected to package-private

```
class Base{protected void run(){}  
class Child extends Base{void run(){}}
```

- A) Valid
- B) Compile error due to reduced visibility
- C) Runtime error
- D) Becomes public

Answer: B) Compile error due to reduced visibility

Explanation: The child method narrows protected to package-private.

Q72. protected to public

```
class Base{protected void run(){}  
class Child extends Base{public void run(){}}
```

- A) Invalid
- B) Valid override
- C) Runtime error
- D) Becomes protected

Answer: B) Valid override

Explanation: Overriding may broaden visibility.

Q73. public to protected

```
class Parent{public void show(){}  
class Child extends Parent{protected void show(){}}
```

- A) Valid
- B) Compile error
- C) Runtime error
- D) Private

Answer: B) Compile error

Explanation: An override cannot reduce public visibility.

Q74. public/protected rule

```
class Parent{protected void show(){}  
class Child extends Parent{public void show(){}}
```

- A) Compile error
- B) Valid override
- C) Runtime error
- D) Becomes protected

Answer: B) Valid override

Explanation: protected→public broadens visibility.

Q75. Serializable

```
class Employee implements Serializable{
    int id; String name;
}
```

- A) Can participate in serialization
- B) Becomes abstract
- C) Becomes immutable
- D) Becomes String

Answer: A) Can participate in serialization

Explanation: Serializable marks the class for Java serialization.

Q76. transient

```
class Employee implements Serializable{
    int id; String name; transient String password;
}
```

- A) Password persisted
- B) Password excluded from default serialization
- C) Password becomes text 'transient'
- D) Error

Answer: B) Password excluded from default serialization

Explanation: transient excludes the field from default serialization.

Q77. deserialized transient output

```
Employee e=new Employee(101,"Mira","secret123");
// serialize and deserialize e
System.out.println(restored.name);
System.out.println(restored.password);
```

- A) Mira / secret123
- B) Mira / null
- C) null / secret123
- D) null / null

Answer: B) Mira / null

Explanation: name is serialized; transient password is not persisted by default.

Q78. serialVersionUID

```
class Employee implements Serializable{
    private static final long serialVersionUID=1L;
}
```

- A) Employee id
- B) Serialized class version compatibility checking
- C) Encryption key
- D) Prevents serialization

Answer: B) Serialized class version compatibility checking

Explanation: serialVersionUID is used for compatibility checking of serialized class versions.

Q79. FileOutputStream

```
FileOutputStream fos=new FileOutputStream("data.bin");
fos.write(new byte[]{65,66,67}); fos.close();
```

- A) Raw bytes
- B) Formatted text
- C) Regex
- D) Inheritance

Answer: A) Raw bytes

Explanation: FileOutputStream writes raw byte data.

Q80. *PrintWriter*

```
PrintWriter w=new PrintWriter("report.txt");  
w.println("Sales Report");  
w.printf("Revenue: %.2f%n",125000.50);  
w.close();
```

- A) Raw binary
- B) Formatted character/text
- C) Hashing
- D) Regex

Answer: B) Formatted character/text

Explanation: PrintWriter is useful for formatted text output.

Q81. *byte vs character output*

```
// Choose the correct pairing:  
FileOutputStream / PrintWriter
```

- A) raw bytes / formatted text
- B) formatted text / raw bytes
- C) both raw bytes
- D) both regex

Answer: A) raw bytes / formatted text

Explanation: This is the exercise's byte-vs-character output distinction.

Q82. *polymorphism with two subclasses*

```
Animal a;  
a=new Dog(); a.speak();  
a=new Cat(); a.speak();
```

- A) Animal / Animal
- B) Dog / Cat
- C) Cat / Dog
- D) Animal / Dog

Answer: B) Dog / Cat

Explanation: The actual object changes from Dog to Cat.

Q83. *super method*

```
class Animal{void speak(){System.out.println("Animal");}}  
class Dog extends Animal{  
void speak(){super.speak();System.out.println("Dog");}  
}  
new Dog().speak();
```

- A) Dog
- B) Animal / Dog
- C) Dog / Animal
- D) Error

Answer: B) Animal / Dog

Explanation: super.speak() explicitly calls the superclass implementation.

Q84. *super vs this*

```
// In a child class:  
super.run();
```

- A) Current object only
- B) Superclass implementation/context
- C) Static class
- D) Interface only

Answer: B) Superclass implementation/context

Explanation: super refers to superclass behavior; this refers to the current object.

Q85. abstract + polymorphism

```
abstract class BaseService{
    abstract void run();
    final void policy(){System.out.println("Policy");}
}
class ReportService extends BaseService{
    void run(){System.out.println("Run");}
}
BaseService s=new ReportService();
s.run(); s.policy();
```

A) Base / Policy

B) Run / Policy

C) Run / Error

D) Compile error

Answer: B) Run / Policy

Explanation: run() dispatches to ReportService; policy() is inherited final behavior.

Exercise 4 — Streams, Resources, Date/Time, Files & HTTP

Q86. filter + count output

```
List<Integer> n=Arrays.asList(1,2,3,4,5);  
long c=n.stream().filter(x->x>2).count();  
System.out.println(c);
```

- A) 2
- B) 3
- C) 5
- D) Error

Answer: B) 3

Explanation: 3, 4, and 5 satisfy the filter.

Q87. terminal operation

```
numbers.stream().filter(n->n>2).count();
```

- A) stream()
- B) filter()
- C) count()
- D) predicate

Answer: C) count()

Explanation: count() is terminal; filter() is intermediate.

Q88. no terminal operation

```
numbers.stream().filter(n->n>2);
```

- A) Final result immediately
- B) Terminal operation triggers processing
- C) count auto-runs
- D) Error

Answer: B) Terminal operation triggers processing

Explanation: The exercise emphasizes terminal operations trigger stream processing.

Q89. map + reduce

```
List<Integer> n=Arrays.asList(1,2,3,4,5);  
int t=n.stream().map(x->x*2).reduce(0,Integer::sum);  
System.out.println(t);
```

- A) 15
- B) 20
- C) 30
- D) 10

Answer: C) 30

Explanation: The mapped values are 2,4,6,8,10; sum is 30.

Q90. modified map + reduce

```
List<Integer> n=Arrays.asList(1,2,3);  
int t=n.stream().map(x->x+10).reduce(0,Integer::sum);  
System.out.println(t);
```

- A) 6
- B) 16
- C) 33
- D) 36

Answer: D) 36

Explanation: 11+12+13 = 36.

Q91. *reduce identity*

```
List<Integer> n=Arrays.asList(1,2,3);
int t=n.stream().reduce(10,Integer::sum);
System.out.println(t);
```

- A) 6
- B) 10
- C) 16
- D) 13

Answer: C) 16

Explanation: The identity 10 is included: 10+1+2+3.

Q92. *try-with-resources*

```
try(BufferedReader br=Files.newBufferedReader(path)){
    System.out.println(br.readLine());
}
```

- A) Prevents compile
- B) Automatically closes resource
- C) Converts bytes to int
- D) Creates thread

Answer: B) Automatically closes resource

Explanation: try-with-resources closes compatible resources automatically.

Q93. *BufferedReader resource*

```
try(BufferedReader br=Files.newBufferedReader(path)){ }
```

- A) It supports AutoCloseable/Closeable
- B) All classes support it
- C) It is a Stream
- D) try accepts anything

Answer: A) It supports AutoCloseable/Closeable

Explanation: BufferedReader supports automatic closing.

Q94. *AutoCloseable*

```
// Which interface is key to try-with-resources?
```

- A) Serializable
- B) AutoCloseable
- C) Runnable
- D) Comparable

Answer: B) AutoCloseable

Explanation: AutoCloseable is the key resource interface.

Q95. *resource after try*

```
try(BufferedReader br=Files.newBufferedReader(path)){
    System.out.println(br.readLine());
}
// execution leaves the block
```

- A) Remains open
- B) Automatically closes
- C) Only becomes null
- D) Is serialized

Answer: B) Automatically closes

Explanation: The resource is closed on leaving the try block.

Q96. *LocalDate*

```
LocalDate d=LocalDate.of(2026,9,30);  
System.out.println(d);
```

- A) Date only
- B) Time only
- C) Date+time+zone
- D) Zone only

Answer: A) Date only

Explanation: LocalDate represents date without time or timezone.

Q97. *LocalDate meaning*

```
LocalDate.of(2026,9,30)
```

- A) Date + current time
- B) Date without time/timezone
- C) Date + UTC
- D) Year only

Answer: B) Date without time/timezone

Explanation: LocalDate is date-only.

Q98. *LocalDateTime*

```
LocalDateTime dt=LocalDateTime.now();
```

- A) Date only
- B) Time only
- C) Date + time, no zone
- D) Date + time + zone

Answer: C) Date + time, no zone

Explanation: LocalDateTime has date and time but no timezone.

Q99. *timezone matters*

```
// Need date + time + timezone information
```

- A) LocalDate
- B) LocalDateTime
- C) ZonedDateTime
- D) String

Answer: C) ZonedDateTime

Explanation: ZonedDateTime is the exercise's type when timezone matters.

Q100. *ZonedDateTime*

```
ZonedDateTime z=ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));  
System.out.println(z);
```

- A) Date only
- B) Time only
- C) Date/time + zone
- D) Zone only

Answer: C) Date/time + zone

Explanation: The value includes Asia/Kolkata timezone information.

Q101. date-time comparison

```
// Compare:  
LocalDate / LocalDateTime / ZonedDateTime
```

- A) date / date+time / date+time+zone
- B) time / zone / date
- C) date+zone / zone / date
- D) identical

Answer: A) date / date+time / date+time+zone

Explanation: This is the core date-time distinction.

Q102. Files.walk

```
try(Stream<Path> files=Files.walk(Paths.get("."))){  
  files.limit(5).forEach(System.out::println);  
}
```

- A) HTTP
- B) Recursive directory traversal
- C) Regex
- D) Serialization

Answer: B) Recursive directory traversal

Explanation: Files.walk traverses below the starting path.

Q103. Files.walk type

```
Stream<Path> files=Files.walk(Paths.get("."));
```

- A) List<Path>
- B) Stream<Path>
- C) Path
- D) File

Answer: B) Stream<Path>

Explanation: The exercise uses Stream<Path> as the result.

Q104. limit(5)

```
Files.walk(Paths.get(".")).limit(5).forEach(System.out::println);
```

- A) Five directory levels
- B) Five stream elements
- C) Deletes five files
- D) Five threads

Answer: B) Five stream elements

Explanation: limit controls the number of stream elements passed onward.

Q105. walk + resource

```
try(Stream<Path> files=Files.walk(Paths.get("."))){  
  files.forEach(System.out::println);  
}
```

- A) Stream/resource can be automatically closed
- B) Converts Path to String
- C) Prevents recursion
- D) Sorts paths

Answer: A) Stream/resource can be automatically closed

Explanation: The walk stream is managed with try-with-resources.

Q106. *HttpClient*

```
HttpClient client=HttpClient.newHttpClient();
```

- A) Local files
- B) HTTP requests/responses
- C) Regex
- D) Serialization

Answer: B) HTTP requests/responses

Explanation: HttpClient is used for HTTP communication.

Q107. *HTTP method*

```
HttpRequest r=HttpRequest.newBuilder()  
.uri(URI.create("https://example.com")).GET().build();
```

- A) POST
- B) PUT
- C) GET
- D) DELETE

Answer: C) GET

Explanation: The builder explicitly calls GET().

Q108. *URI.create*

```
HttpRequest.newBuilder()  
.uri(URI.create("https://example.com"))  
.GET().build();
```

- A) Request URI
- B) Response body
- C) Status code
- D) File path

Answer: A) Request URI

Explanation: URI.create supplies the request URI.

Q109. *HTTP response type*

```
HttpResponse<String> response=  
client.send(request,HttpResponse.BodyHandlers.ofString());
```

- A) HTTP response with String body
- B) Request
- C) File stream
- D) Date object

Answer: A) HTTP response with String body

Explanation: The generic type and body handler indicate a String response body.

Q110. *statusCode()*

```
System.out.println(response.statusCode());
```

- A) URL
- B) HTTP status code
- C) Body
- D) File size

Answer: B) HTTP status code

Explanation: statusCode() returns the HTTP response status.

Q111. HTTP flow

```
HttpClient client=HttpClient.newHttpClient();
HttpRequest request=HttpRequest.newBuilder()
.uri(URI.create("https://example.com")).GET().build();
```

- A) Request→Client→URI
- B) Client→Request→URI→GET
- C) URI→Response→Client
- D) GET→File→Request

Answer: B) Client→Request→URI→GET

Explanation: The conceptual flow is client, request, URI/method, send, response.

Q112. even count stream

```
List<Integer> n=Arrays.asList(1,2,3,4,5);
long r=n.stream().filter(x->x%2==0).count();
System.out.println(r);
```

- A) 2
- B) 3
- C) 5
- D) 1

Answer: A) 2

Explanation: The even values are 2 and 4.

Q113. stream pipeline

```
List<Integer> n=Arrays.asList(1,2,3,4);
long r=n.stream().filter(x->x>1).map(x->x*10).count();
```

- A) 20
- B) 30
- C) 3
- D) 4

Answer: C) 3

Explanation: After filtering there are 3 elements; map changes values but not count.

Q114. map and original list

```
List<Integer> n=Arrays.asList(1,2,3);
List<Integer> r=n.stream().map(x->x*2).toList();
System.out.println(n); System.out.println(r);
```

- A) [2,4,6] / [2,4,6]
- B) [1,2,3] / [2,4,6]
- C) [1,2,3] / [1,2,3]
- D) Error

Answer: B) [1,2,3] / [2,4,6]

Explanation: The transformed stream produces a separate result; the source list is unchanged.

Q115. big stream pipeline

```
List<Integer> n=Arrays.asList(1,2,3,4,5);
int r=n.stream().filter(x->x>2).map(x->x*2)
.reduce(0,Integer::sum);
System.out.println(r);
```

- A) 12
- B) 18
- C) 24
- D) 30

Answer: C) 24

Explanation: Filter gives 3,4,5; map gives 6,8,10; reduce sums to 24.

Final Rapid Revision

Exercise 1: `int`→`long` = widening; `int`→`Integer` = boxing; `this(...)` calls another constructor and must be first; `static` belongs to class; `final` cannot be reassigned; `false &&` skips right side; `true ||` skips right side; `*` before `+`.

Exercise 2: `List` allows duplicates; `Set` does not; `HashSet/HashMap` depend on `equals()+hashCode()`; `? extends Number` is safe to read as `Number` but not to add a specific value; `Iterator.remove()` removes the last `next()` element; `find()` searches within input; `matches()` checks the whole input.

Exercise 3: `Animal a=new Dog()` is runtime polymorphism; `super` accesses superclass behavior; inheritance = IS-A; composition = HAS-A; one superclass + multiple interfaces; `final` method cannot be overridden; overriding cannot reduce visibility; `transient` is excluded from default serialization; `FileOutputStream` = bytes; `PrintWriter` = text.

Exercise 4: `filter/map` are intermediate; `count/reduce/forEach` are terminal; terminal operations trigger stream processing; `try-with-resources` closes `AutoCloseable` resources; `LocalDate=date`; `LocalDateTime=date+time`; `ZonedDateTime=date+time+zone`; `Files.walk()` gives `Stream`; `HttpClient` handles HTTP requests/responses.